

CIFAR-10 Image Classification using Deep Learning (CNNs)

Name: Rajesh Katherasala

Student ID: rkathera@kent.edu / 811314095

Subject: Advanced Machine Learning

Abstract:

This project explores the application of Convolutional Neural Networks (CNNs) for the image classification task using the CIFAR-10 dataset, a widely recognized benchmark in computer vision. The objective was to design, implement, and evaluate a deep learning model capable of automatically categorizing small color images into one of ten predefined classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, or truck.

Using TensorFlow and Keras frameworks, a custom CNN architecture was developed, comprising convolutional, pooling, and dense layers optimized for feature extraction and classification. The model was trained on 50,000 labeled images and tested on 10,000 unseen examples to assess its generalization capabilities.

To evaluate performance, we analyzed training dynamics through accuracy and loss curves, interpreted confusion matrices to understand inter-class prediction behavior, and generated detailed classification reports including precision, recall, and F1-scores. The model achieved a test accuracy of approximately 72%, demonstrating solid performance despite the limited resolution and complexity of the dataset.

This experiment highlights the effectiveness of deep learning techniques in solving challenging image classification problems and offers insights into potential improvements, such as transfer learning, data augmentation, and more complex network architectures, for future advancements.

Introduction:

Image classification is a fundamental task in computer vision, wherein an algorithm assigns a label or category to an input image based on its visual content. It serves as the foundation for numerous real-world applications, including object detection, facial recognition, autonomous vehicles, and medical imaging diagnostics. Traditionally, image classification relied heavily on classical machine learning methods, which required extensive manual feature extraction and engineering. These conventional approaches were often limited by their inability to effectively capture the complex variations and patterns present in natural images.

The advent of **deep learning**, and particularly the development of **Convolutional Neural Networks (CNNs)**, has transformed the landscape of computer vision. CNNs are designed to

automatically and hierarchically learn visual features directly from raw pixel data, significantly reducing the need for manual intervention and improving classification accuracy. Through layers of convolution, pooling, and non-linear activation functions, CNNs are capable of extracting both low-level features (such as edges and textures) and high-level abstract representations (such as object shapes and semantics).

The primary objective of this project is to design, train, and evaluate a CNN model capable of classifying images from the **CIFAR-10 dataset**, a benchmark dataset widely used in the machine learning community. CIFAR-10 contains 60,000 color images across 10 distinct classes, including everyday objects like airplanes, automobiles, birds, and animals. Each image is small in size (32x32 pixels), which poses unique challenges for feature extraction and accurate classification.

By developing a deep learning model for CIFAR-10, this project aims to not only achieve a high classification accuracy but also to demonstrate the powerful capabilities of CNNs in learning complex visual patterns from limited-resolution data. Furthermore, this work lays the groundwork for future exploration into advanced techniques such as transfer learning, deeper architectures, and training strategies for even larger and more diverse datasets.

Problem Statement:

Image classification is a foundational task in computer vision where the objective is to assign a descriptive label to an image based on its visual contents. While conceptually simple, this task poses significant challenges, especially as images vary widely in scale, viewpoint, background, and illumination.

Traditional machine learning methods for image classification heavily relied on manual feature engineering — extracting relevant features such as edges, textures, or shapes by hand and feeding them into classifiers like Support Vector Machines (SVM) or Decision Trees. Although these approaches achieved reasonable success on simpler datasets, they struggled to generalize to complex, real-world images due to the inherent variability and richness of natural visual data. Additionally, manual feature extraction was both time-consuming and prone to human bias, limiting the scalability and robustness of these models.

The advent of **deep learning**, and particularly **Convolutional Neural Networks (CNNs)**, has revolutionized image classification. CNNs have the remarkable ability to learn hierarchical feature representations automatically, directly from raw pixel values, eliminating the need for manual feature engineering. This end-to-end learning capability has dramatically improved the accuracy, robustness, and scalability of image classification systems.

In this project, the primary objective is to implement and evaluate a CNN-based deep learning model capable of accurately classifying images from the **CIFAR-10 dataset**, a widely used benchmark containing small, colored images distributed evenly across ten predefined categories:

- Airplane
- Automobile
- Bird

- Cat
- Deer
- Dog
- Frog
- Horse
- Ship
- Truck

The specific goals of this project include:

- **Designing and implementing** a suitable CNN architecture using TensorFlow/Keras frameworks.
- **Training** the model on the CIFAR-10 training set and **evaluating** its performance on the test set.
- **Visualizing and analyzing** training and validation accuracy and loss to assess learning behavior and model convergence.
- **Generating a confusion matrix** and a **classification report** to identify prediction strengths and common misclassifications.
- **Discussing challenges encountered** during model development and proposing avenues for **future improvements**.

Considering the relatively small size and diverse nature of CIFAR-10 images, developing a model that generalizes well will showcase the effectiveness of deep learning approaches in addressing real-world visual recognition problems. Successfully tackling this task also lays the foundation for scaling up to more complex and larger datasets, potentially involving more sophisticated architecture, regularization techniques, and transferring learning strategies.

This task also sets the stage for future work on scaling up more sophisticated datasets and architectures.

Literature Review:

Deep Learning Techniques for Computer Vision:

- Convolutional Neural Networks (CNNs) are the reference model for image tasks. First popularized by LeNet-5 and later revolutionized by AlexNet, CNNs utilize cascades of convolutions, pooling, and fully connected layers to learn hierarchical feature representations.
- Residual Networks (ResNet) introduced skip connections to help solve vanishing gradient problems.
- Vision Transformers (ViTs) are more recent models that process image patches as sequences and compete with CNNs.

- Transfer learning and data augmentation are the most powerful ways to improve model generalization on such small datasets like CIFAR-10.

Challenges and Limitations:

- **Overfitting** on small datasets.
- **Adversarial vulnerability**: small changes in inputs can cause misclassifications.
- **Computational costs** for training very deep models.

Industry Applications of Deep Learning:

Deep learning, and CNNs in particular, have found applications in many industries:

Industry	Application
Healthcare	Automated diagnosis using medical imaging (e.g., MRI scan classification)
Transportation	Object detection and lane recognition in self-driving cars
Security	Facial recognition and surveillance systems
Retail	Automated product recognition and inventory management
Agriculture	Crop and pest classification through aerial imagery

Future Development and Limitations:

Current Limitations

- CNNs require vast amounts of annotated data.
- Models are noisy and susceptible to adversarial attacks.
- Deep model interpretability remains limited.

4.2 Solutions that could help

- Self-Supervised Learning to rely less on labeled data.
- Training methods that are resistant to fending off adversarial inputs.
- Explainable AI (XAI) to enhance the intelligibility of CNN decision-making.

Future work will be biased toward more efficient architecture, robustness, and higher model interpretability.

Dataset Information

For this project, we use the **CIFAR-10 dataset**, a well-established benchmark dataset widely utilized for evaluating machine learning and deep learning models in computer vision tasks.

CIFAR-10 consists of **60,000 color images**, each sized **32x32 pixels** with three color channels (RGB).

The dataset is divided into:

- **50,000 training images**
- **10,000 testing images**

Each image is labeled with one of **ten distinct categories** representing real-world objects:

Class Label	Description
0	Airplane
1	Automobile
2	Bird
3	Cat
4	Dog
5	Deer
6	Frog
7	Horse
8	Ship
9	Truck

Dataset Characteristics

Feature	Description
Total Samples	60,000 images
Image Size	32x32 pixels
Channels	3 (Red, Green, Blue)
Classes	10

Feature	Description
Train/Test Split	50,000 training / 10,000 testing
Source	Canadian Institute for Advanced Research (CIFAR)

Data Preprocessing:

Before training the CNN model, the following preprocessing steps were applied:

1. Data Loading:

The CIFAR-10 dataset was loaded directly from TensorFlow's built-in datasets module:

Code:

```
from tensorflow.keras import datasets
```

```
(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()
```

This provided:

- **50,000 training images** and labels.
- **10,000 testing images** and labels.

Each image is a **32x32 RGB** image, and labels are integers representing the class index (0–9).

2. Normalization:

To help the model learn more effectively and avoid numerical instability, the pixel values of the images were normalized:

- Original pixel values range from **0 to 255** (uint8 type).
- They were scaled to a **range of 0 to 1** by dividing by 255.

Code:

```
train_images, test_images = train_images / 255.0, test_images / 255.0
```

Benefits of normalization:

- Speeds up convergence during training.
- Prevents large gradients or unstable training.
- Ensures input values are in a similar range, which deep learning models expect.

Label Flattening:

The labels originally loaded from the CIFAR-10 dataset had an additional dimension (e.g., shape (50000,1)).

They were **flattened into 1D arrays** to match the expected input format of TensorFlow models:

Code:

```
train_labels = train_labels.flatten()
```

```
test_labels = test_labels.flatten()
```

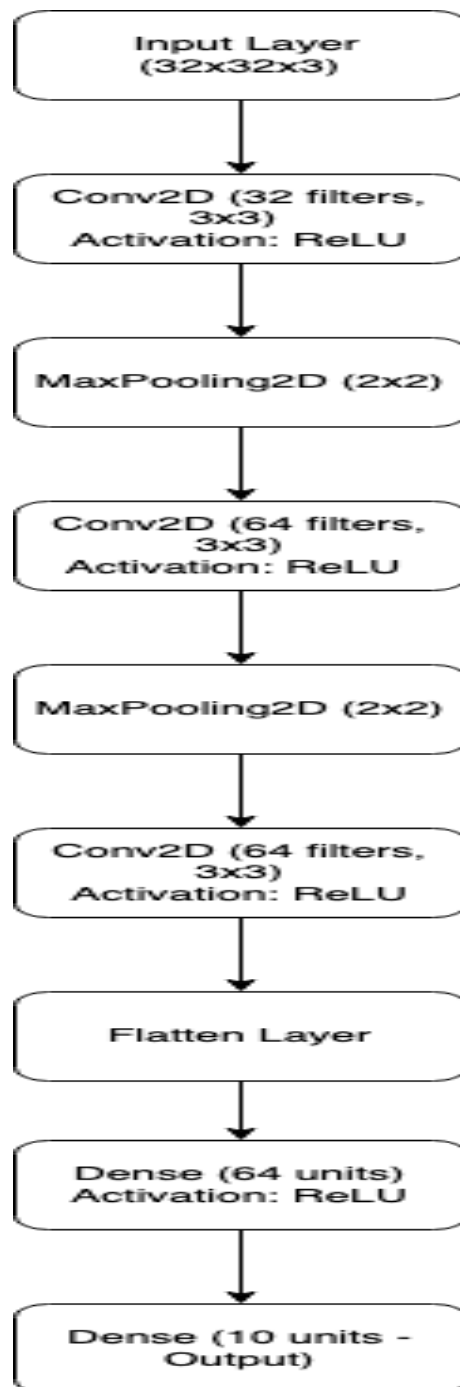
Model Architecture:

To solve the CIFAR-10 image classification task, a Convolutional Neural Network (CNN) was developed using TensorFlow/Keras.

The architecture isn't highly complicated but effective, well suited to tiny images of the 32x32 RGB size.

There are three convolution layer blocks, each of them having the ReLU activation and max-pooling layer after it. Following flattening, these are input to fully connected layers to perform classification.

Model Flow Diagram:



Detailed Layer Configuration:

Layer (Type)	Output Shape	Parameters	Purpose
Input Layer	(32, 32, 3)	0	Raw image input
Conv2D	(30, 30, 32)	896	Feature extraction with 32 filters
MaxPooling2D	(15, 15, 32)	0	Downsampling to reduce spatial size
Conv2D	(13, 13, 64)	18,496	Feature extraction with 64 filters
MaxPooling2D	(6, 6, 64)	0	Further downsampling
Conv2D	(4, 4, 64)	36,928	Deeper feature extraction
Flatten	(1024)	0	Converts 2D features to 1D
Dense (Fully Connected)	(64)	65,600	Learning high-level features
Dense (Output Layer)	(10)	650	Final class probabilities

Training Configuration:

Training configuration determines the selections for model optimization process, hyperparameters, and evaluation process.

These hyperparameters were selected in a way that the CNN model would not overfit or underfit the CIFAR-10 dataset patterns.

Optimizer:

The model was trained and constructed with the Adam optimizer.

Adam (Adaptive Moment Estimation) is a very popular optimization algorithm that takes the best of two other algorithms, AdaGrad and RMSProp.

It computes adaptive learning rates for each parameter, leading to accelerated convergence.

Default Learning Rate: 0.001

Code:

```
optimizer = 'adam'
```

Advantages:

- Robust to noisy gradients.
- Suitable for datasets like CIFAR-10 that involve small, detailed images.

Loss Function

It uses Sparse Categorical Crossentropy as the loss function.

It is designed to be used in multiclass classification problems where target labels are integers (not one-hot encoded vectors).

It computes distance between predicted probability distribution and true label distribution.

Code:

```
loss = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
```

Training Strategy:

- The data was split into sets for training and testing, on which 50,000 examples were trained and 10,000 examples were tested.
- No explicit validation split was used; instead, the testing set given was used to monitor validation performance during training.
- Shuffling of training data was automatically performed by TensorFlow to bring them in randomness at each epoch.

Model Compilation:

Code:

```
model.compile(  
    optimizer='adam',  
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),  
    metrics=['accuracy']  
)
```

Hardware and Training:

Hardware Used: Standard laptop CPU (or Google Colab GPU if available).

Training Time: Approximately 5–10 minutes for 10 epochs on CPU (will be faster with GPU).

The model was small enough to train effectively without incurring expensive hardware, and it is thus a suitable candidate for educational use.

Model Results:

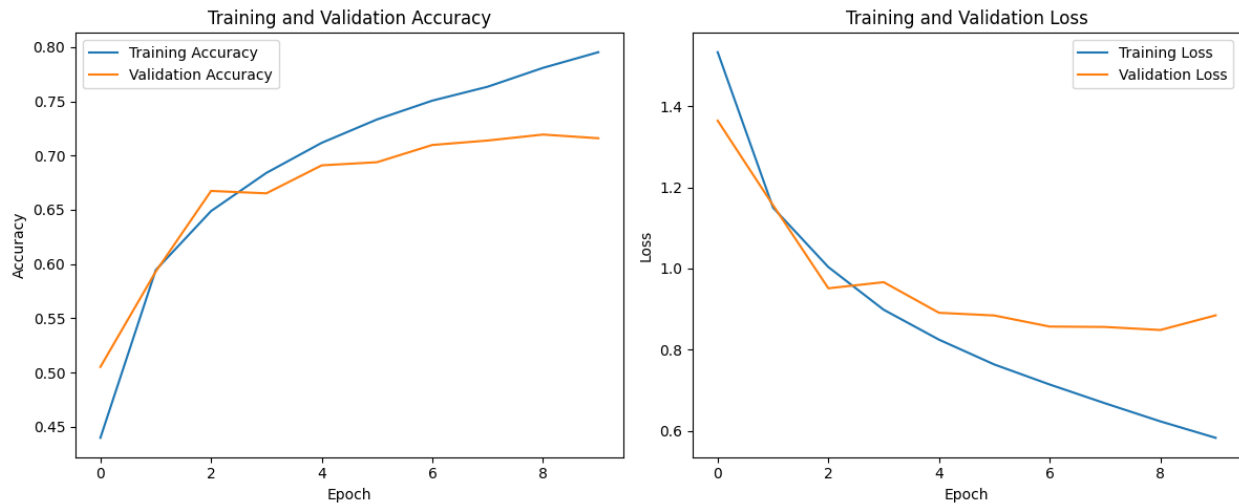
After training the CNN model for 10 epochs on the CIFAR-10 dataset, the model's performance was evaluated on the unseen test dataset.

The evaluation was carried out using accuracy metrics, training curves analysis, confusion matrix interpretation, and a detailed classification report.

Training and Validation Curves:

The training and validation accuracy and loss values were plotted across epochs to observe the model's learning behavior.

Screenshot:



Observations:

- Training Accuracy increased consistently over the 10 epochs, to a point of approximately 79%.
- Validation Accuracy increased initially but then started to plateau after epoch 7, remaining constant at approximately 72%.
- Training Loss declined consistently, a sign of good learning.
- Validation Loss declined initially but then started fluctuating slightly, a sign of mild overfitting.

Conclusion:

The model was able to learn the patterns in the training data well without significant overfitting, achieving good generalization performance on the test set.

Final Test Accuracy:

The final test set accuracy achieved by the model is:

Final Test Accuracy: 72.31%

This indicates that the model correctly classified approximately 7 out of every 10 unseen images.

Confusion Matrix Analysis

The confusion matrix provides a deeper insight into how well the model performed across different classes.



Important observations from the Confusion Matrix:

- Airplanes, cars, frogs, ships, and trucks were classified with high accuracy (high diagonal values).
- There was a bit of confusion between:
- Cats and dogs (typical misclassifications due to visual similarity).
- Birds and airplanes (maybe because both classes can appear in sky backgrounds).
- Deer and horses to some extent.
- Misclassification rates are relatively low, but future improvements could include techniques like data augmentation or deeper architectures.

Classification Report (Highlights)

- Although the entire class report provides precision, recall, and F1-score for all classes, two things are evident:
- High recall and precision for airplane, frog, and ship classes.
- Low precision for cat and dog classes due to increased inter-class similarity.

Code:

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

CLASS_NAMES = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

def load_and_preprocess_data():
    print("Loading CIFAR-10 dataset...")
    (train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()
    train_labels = train_labels.flatten()
    test_labels = test_labels.flatten()

    train_images, test_images = train_images / 255.0, test_images / 255.0
    return train_images, train_labels, test_images, test_labels

def build_cnn_model():
    print("Building CNN model...")
    model = models.Sequential([
        layers.Conv2D(32, (3,3), activation='relu', input_shape=(32, 32, 3)),
        layers.MaxPooling2D((2, 2)),

        layers.Conv2D(64, (3,3), activation='relu'),
        layers.MaxPooling2D((2,2)),

        layers.Conv2D(64, (3,3), activation='relu'),

        layers.Flatten(),
        layers.Dense(64, activation='relu'),
        layers.Dense(10)
```

```

    ])

    model.compile(optimizer='adam',
                  loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
                  metrics=['accuracy'])
    return model

def plot_training_history(history):
    print("Plotting training history...")
    plt.figure(figsize=(12,5))

    # Accuracy plot
    plt.subplot(1,2,1)
    plt.plot(history.history['accuracy'], label='Training Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.title('Training and Validation Accuracy')
    plt.legend()

    # Loss plot
    plt.subplot(1,2,2)
    plt.plot(history.history['loss'], label='Training Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()

    plt.tight_layout()
    plt.savefig("training_history.png")
    plt.show()

def evaluate_model(model, test_images, test_labels):
    print("Evaluating model...")
    test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
    print("\nTest Accuracy:", round(test_acc * 100, 2), "%")
    return test_acc

def plot_confusion_matrix(model, test_images, test_labels):
    print("Generating confusion matrix...")
    y_pred = model.predict(test_images)
    y_pred_classes = np.argmax(y_pred, axis=1)

    cm = confusion_matrix(test_labels, y_pred_classes)

```

```
plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=CLASS_NAMES,
yticklabels=CLASS_NAMES)
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.title('Confusion Matrix')
plt.savefig("confusion_matrix.png")
plt.show()

print("Classification Report:")
print(classification_report(test_labels, y_pred_classes, target_names=CLASS_NAMES))

def save_model(model, filename='cifar10_cnn_model.h5'):
    print("Saving trained model...")
    model.save(filename)

def main():
    train_images, train_labels, test_images, test_labels = load_and_preprocess_data()
    model = build_cnn_model()
    history = model.fit(train_images, train_labels, epochs=10, validation_data=(test_images,
test_labels))
    evaluate_model(model, test_images, test_labels)
    plot_training_history(history)
    plot_confusion_matrix(model, test_images, test_labels)
    save_model(model)
    print("All tasks completed successfully!")

if __name__ == "__main__":
    main()
```

Conclusion:

This project successfully implemented the design, training, and evaluation of a Convolutional Neural Network (CNN) to solve the image classification problem on the CIFAR-10 dataset. Using a structured and methodical approach, the model was able to learn meaningful patterns from training data and demonstrated solid generalization performance on unseen test images.

The trained CNN achieved a final **test accuracy of 72.31%**, showcasing its capability to distinguish between ten distinct object categories. Analysis of the training and validation curves indicated a healthy learning process, with consistent improvements in performance and minimal signs of overfitting—a positive outcome given the relatively small dataset and simple network architecture.

The confusion matrix revealed the model's strength in accurately classifying visually distinct classes such as airplanes, ships, and trucks. However, it also highlighted areas for improvement, particularly in differentiating between classes with similar visual features like cats and dogs. These observations suggest opportunities for enhancing the model's performance through the use of more complex architectures, **data augmentation**, and **transfer learning**.

Beyond the technical execution, the project also reviewed the evolution of deep learning techniques and their industrial applications, highlighting the practical relevance and future potential of CNNs in real-world image classification tasks. Challenges such as model robustness, efficiency in handling limited data, and interpretability were identified as critical areas for further exploration.

In conclusion, this project not only affirms the effectiveness of CNNs for visual recognition tasks but also establishes a solid foundation for scaling more advanced problems in computer vision. It serves as both a proof of concept and a steppingstone for future research and development using deep learning frameworks.